

1. What Is a Variable?

A variable is a named reference used to store a value.

```
source_count = 1500
```

Here:

- `source_count` is the variable name.
- `1500` is the stored value.
- `=` is the assignment operator.

Once stored, the value can be reused:

```
print(source_count)  
print(f"Records Received: {source_count}")
```

2. Why Variables Matter in Data Engineering

Variables help store pipeline-related information such as:

- Pipeline name
- Source system
- File path
- Table name
- Record count
- Batch ID
- Retry count
- Pipeline status
- Error message
- Processing duration
- Incremental load date

Instead of repeating values:

```
print("customer_daily_pipeline started")  
print("customer_daily_pipeline completed")
```

Store the value once:

```
pipeline_name = "customer_daily_pipeline"
```

```
print(f'{pipeline_name} started')  
print(f'{pipeline_name} completed')
```

This makes code easier to update and maintain.

3. Main Python Data Types

The main data types covered in this class are:

Data Type	Python Type	Example
String	<code>str</code>	<code>"customer_ap i"</code>
Integer	<code>int</code>	<code>1500</code>
Float	<code>float</code>	<code>24.6</code>
Boolean	<code>bool</code>	<code>True</code>
No value	<code>NoneType</code>	<code>None</code>

4. String

A string stores text and must be written inside quotation marks.

```
pipeline_name = "customer_daily_pipeline"  
source_system = "customer_api"  
target_table = "silver_customers"  
file_name = "customers_2026_07_30.csv"
```

Strings are commonly used for:

- File names
- File paths
- Table names

- Column names
- API endpoints
- SQL queries
- Pipeline names
- Status messages
- Error messages

Example:

```
source_path = "/data/landing/customers.csv"  
pipeline_status = "RUNNING"
```

Even though a file path represents a location, Python stores it as text.

5. Integer

An integer stores a whole number without a decimal point.

```
source_count = 1500  
target_count = 1475  
retry_count = 3  
batch_id = 101
```

Integers are commonly used for:

- Record counts
- Retry counts
- Batch IDs
- File counts
- Failure counts
- Page numbers
- Number of partitions

Example:

```
rejected_count = source_count - target_count  
  
print(rejected_count)
```

Output:

6. Float

A float stores a number containing a decimal point.

```
load_duration = 24.6
file_size_mb = 15.75
data_quality_percentage = 98.5
```

Floats are commonly used for:

- Pipeline duration
- File size
- Percentage values
- Data-quality scores
- Average processing time
- Monetary values

Example:

```
source_count = 1500
valid_count = 1475

data_quality_percentage = valid_count / source_count * 100

print(f"Data Quality Percentage: {data_quality_percentage:.2f}%")
```

Output:

Data Quality Percentage: 98.33%

`:.2f` displays the value using two digits after the decimal point.

7. Boolean

A Boolean contains only two possible values:

True
False

The first letter must be capitalised.

Examples:

```
pipeline_successful = False  
file_exists = True  
schema_valid = True
```

Boolean values are commonly used to represent conditions.

```
source_count = 1500  
target_count = 1475  
  
counts_match = source_count == target_count  
  
print(counts_match)
```

Output:

False

The comparison produces a Boolean result.

8. None

None represents the absence of a value.

```
error_message = None  
last_processed_date = None
```

Possible meanings:

- No error has occurred.
- The value is not yet available.
- The pipeline has never run before.
- No date has been assigned.

None vs Empty String

```
error_message = None  
empty_message = ""
```

These are not the same.

- `None` means no value.
- `""` is a string containing zero characters.

None vs "None"

```
error_message = None  
text_value = "None"
```

- `None` has the type `NoneType`.
 - `"None"` is a string.
-

9. Checking Data Types

Use the `type()` function to check how Python is storing a value.

```
pipeline_name = "customer_daily_pipeline"  
source_count = 1500  
load_duration = 24.6  
pipeline_successful = False  
error_message = None
```

```
print(type(pipeline_name))  
print(type(source_count))  
print(type(load_duration))  
print(type(pipeline_successful))  
print(type(error_message))
```

Output:

```
<class 'str'>  
<class 'int'>  
<class 'float'>
```

```
<class 'bool'>  
<class 'NoneType'>
```

Why `type()` Is Useful

Source systems may provide values in the wrong format.

```
source_count = "1500"  
  
print(type(source_count))
```

Output:

```
<class 'str'>
```

Although the value looks numeric, quotation marks make it a string.

10. Dynamic Typing

Python is dynamically typed.

This means the data type does not need to be declared separately.

```
source_count = 1500
```

Python automatically identifies it as an integer.

A variable can later be reassigned to another type:

```
source_count = 1500  
source_count = "1500"
```

This is allowed, but unnecessary type changes should be avoided because they can cause pipeline errors.

Keep a variable's meaning and data type consistent.

11. Variable Naming Rules

Valid Characters

Variable names can contain:

- Letters
- Numbers
- Underscores

```
source_count = 1500  
source_count_1 = 1500
```

Cannot Start with a Number

Incorrect:

```
1_source_count = 1500
```

Correct:

```
source_count_1 = 1500
```

Spaces Are Not Allowed

Incorrect:

```
source count = 1500
```

Correct:

```
source_count = 1500
```

Avoid Special Characters

Incorrect:


```
source-count = 1500  
source@count = 1500
```

Correct:

```
source_count = 1500
```

Variable Names Are Case-Sensitive

```
source_count = 1500  
Source_Count = 2000  
SOURCE_COUNT = 3000
```

Python treats these as three different variables.

Python Keywords Cannot Be Used

Do not use reserved words such as:

```
if  
for  
class  
return  
True  
False  
None
```

12. Recommended Naming Style

Python commonly uses **snake case**.

Snake case means:

- Lowercase letters
- Words separated by underscores

Recommended:

```
pipeline_name = "customer_pipeline"  
source_record_count = 1500  
last_processed_date = None  
data_quality_percentage = 98.5
```

Avoid unclear names:

```
a = 1500  
b = 1475  
c = a - b
```

Prefer:

```
source_count = 1500  
target_count = 1475  
rejected_count = source_count - target_count
```

Meaningful names make code easier to understand, debug and maintain.

13. Constants

A constant is a value that should normally remain unchanged during program execution.

Python does not technically prevent constants from changing, but uppercase names indicate that the value should remain fixed.

```
MAX_RETRY_COUNT = 3  
DEFAULT_TIMEOUT = 30  
SUPPORTED_FILE_FORMAT = "csv"
```

Constants may be used for:

- Maximum retries
 - Default timeout
 - Default batch size
 - Supported extensions
 - Environment names
-

14. Assigning and Updating Variables

Multiple Assignment

```
source_count, target_count = 1500, 1475
```

For important pipeline metadata, separate lines may be easier to read.

```
source_count = 1500  
target_count = 1475
```

Same Value to Multiple Variables

```
source_status = target_status = "PENDING"
```

Use this only when both variables genuinely require the same value.

Updating a Variable

```
pipeline_status = "STARTED"  
pipeline_status = "RUNNING"  
pipeline_status = "COMPLETED"
```

Pipeline values can change during execution.

Incrementing a Count

```
processed_count = 0  
  
processed_count += 100  
processed_count += 200  
  
print(processed_count)
```

Output:

300

15. Pipeline Metadata Example

```
pipeline_name = "customer_daily_pipeline"
source_system = "customer_api"
target_table = "silver_customers"

source_count = 1500
target_count = 1475
rejected_count = source_count - target_count

load_duration_seconds = 24.6
pipeline_successful = source_count == target_count
error_message = None

print("Pipeline Metadata")
print("-----")
print(f"Pipeline Name: {pipeline_name}")
print(f"Source System: {source_system}")
print(f"Target Table: {target_table}")
print(f"Source Count: {source_count}")
print(f"Target Count: {target_count}")
print(f"Rejected Count: {rejected_count}")
print(f"Load Duration: {load_duration_seconds} seconds")
print(f"Pipeline Successful: {pipeline_successful}")
print(f"Error Message: {error_message}")
```

16. Common Beginner Mistakes

Missing Quotation Marks

Incorrect:

```
pipeline_name = customer_pipeline
```

Correct:

```
pipeline_name = "customer_pipeline"
```

Lowercase Boolean Values

Incorrect:

```
pipeline_successful = true
```

Correct:

```
pipeline_successful = True
```

Writing None Inside Quotes

```
error_message = None    # No value  
error_message = "None"  # String
```

Assuming a Numeric-Looking Value Is Numeric

```
source_count = "1500"
```

This is a string because it is inside quotation marks.

Using Meaningless Names

Avoid:

```
x = "customer_api"  
y = 1500  
z = False
```

Prefer:

```
source_system = "customer_api"  
source_count = 1500  
pipeline_successful = False
```

17. Important Revision Points

- Variables store reusable pipeline information.
 - Python automatically identifies the data type.
 - Strings contain text.
 - Integers contain whole numbers.
 - Floats contain decimal values.
 - Booleans contain `True` or `False`.
 - `None` represents the absence of a value.
 - `type()` helps verify how a value is stored.
 - A value inside quotation marks is always a string.
 - Python is dynamically typed.
 - Variable names should follow snake case.
 - Meaningful names improve maintainability.
 - Uppercase names are commonly used for constants.
-

18. Practice Assignment

Create variables for the following order pipeline:

Pipeline Name: `daily_order_pipeline`

Source System: `order_api`

Target Table: `silver_orders`

Source Count: 2500

Target Count: 2470

Load Duration: 42.8 seconds

Pipeline Successful: `False`

Error Message: No value

Complete the following tasks:

1. Create one variable for every value.
2. Use meaningful snake-case names.
3. Calculate the rejected record count.
4. Print all values using f-strings.
5. Print the data type of every variable using `type()`.

Expected rejected count:

30

19. Practice Challenge

Add the following values:

Retry Count: 2

File Size: 35.75 MB

File Received: True

Last Error Time: No value

Choose the correct data type for every value and print its type.

20. Interview-Relevant Questions

1. What is a variable in Python?

A variable is a named reference used to store a value. In Data Engineering, variables store information such as file paths, table names, record counts, pipeline statuses and execution details.

2. What are the main basic Python data types?

The main basic types covered here are:

- String
- Integer
- Float
- Boolean
- NoneType

3. What is the difference between an integer and a float?

An integer stores a whole number, while a float stores a decimal number.

Record counts are usually integers, while duration and percentages may be floats.

4. What is the difference between **None** and an empty string?

None means that no value is available.

An empty string is still a string, but it contains zero characters.

5. Is Python statically or dynamically typed?

Python is dynamically typed. The data type is identified automatically from the assigned value.

6. What is the output?

```
source_count = "1500"  
print(type(source_count))
```

Answer:

```
<class 'str'>
```

The value is a string because it is inside quotation marks.

7. Why is **type()** useful in Data Engineering?

It helps verify whether source values have been received in the expected format, especially when reading data from files, APIs or databases.

8. Why should variable types remain consistent?

Changing a variable from one type to another can create unexpected pipeline errors and make code difficult to understand.

9. What is snake-case naming?

Snake case uses lowercase words separated by underscores.

```
source_record_count = 1500
```


10. What is the purpose of uppercase variable names?

Uppercase names are commonly used to indicate constants—values that should normally remain unchanged.

21. Interview Practice: Predict the Output

```
record_count = 1000  
print(type(record_count))
```

```
record_count = "1000"  
print(type(record_count))
```

Expected output:

```
<class 'int'>  
<class 'str'>
```

This is possible because Python is dynamically typed, but frequently changing a variable's type should generally be avoided in clean pipeline code.